

Building GateKeeper: API Management & Control Plane

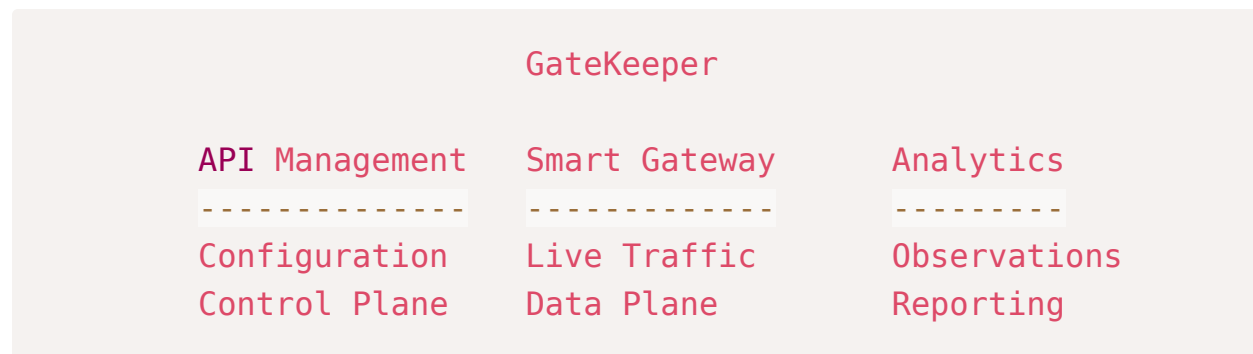
Coder Army

In this part of the GateKeeper project, we start building the **API Management Service**.

GateKeeper consists of three Spring Boot microservices:

```
api-management-service
smart-gateway-service
analytics-service
```

Each service has a very different responsibility:



The goal is not simply to create three Spring Boot applications. The important part is deciding **which problem each service owns**.

1. GateKeeper Architecture

API Management Service

The API Management Service stores the configuration that tells the Gateway **what it should do**.

Examples:

- Which organizations exist?

- Which routes belong to an organization?
- Where should a route forward traffic?
- Which HTTP methods are allowed?
- What is the rate limit?
- What is the response timeout?
- Is a route active?
- Which headers should be added or removed?

This service represents the **Control Plane** of GateKeeper.

Smart Gateway Service

The Smart Gateway handles the actual application traffic.

It reads configuration created through API Management and applies it to incoming requests.

Examples of its future responsibilities include:

- Route matching
- Request forwarding
- Rate limiting
- Header manipulation
- Timeout enforcement
- Idempotency checks

This service represents the **Data Plane**.

Analytics Service

The Analytics Service records and summarizes what happened while traffic passed through GateKeeper.

For example:

- Number of requests

- Response times
- Failed requests
- Rate-limit violations
- Route usage

Its responsibility is primarily **observation and reporting**.

2. Important Terminology

Before designing the database or APIs, we need a common vocabulary.

Organization

A company that uses GateKeeper.

For example:

```
FoodFast
QuickShop
```

In this project, an organization is also our **tenant**.

There is no separate `Tenant` entity.

We generally use:

- `organization` when talking about the database or business model
- `tenant` when talking about ownership and isolation

Management User

A human or machine identity allowed to configure or read GateKeeper.

Examples:

```
platform-admin
foodfast-admin
gateway-service
```

These users belong to GateKeeper itself.

Tenant Customer

A customer of an application using GateKeeper.

For example:

```
FoodFast customer
QuickShop customer
```

A FoodFast customer is **not a GateKeeper management user**.

This distinction becomes extremely important when implementing authentication.

Route

A rule describing how a public API path maps to a backend service.

Example:

```
/foodfast/orders/**
      ↓
FoodFast Order Service
```

A route can also contain policies such as:

- Allowed methods
 - Rate limits
 - Timeouts
 - Header rules
 - Idempotency configuration
-

Control Plane

The part of the system where Gateway configuration is created and managed.

In GateKeeper:

API Management Service = Control Plane

Data Plane

The part of the system through which real application requests travel.

In GateKeeper:

Smart Gateway Service = Data Plane

A simple way to remember the difference:

Control Plane → decides what should happen
Data Plane → makes it happen

3. Understanding Multi-Tenancy

Suppose two companies are using the same GateKeeper installation.

FoodFast

Authentication Service :8091
Restaurant Service :8092
Order Service :8093

QuickShop

Authentication Service :8101
Catalog Service :8102
Order Service :8103

Both companies may expose an `/orders` API.

That immediately creates a problem.

How can one shared Gateway distinguish between:

```
FoodFast /orders
```

and

```
QuickShop /orders
```

One approach is to make the tenant part of the public path.

```
/foodfast/orders/**  
/quickshop/orders/**
```

The first path segment now identifies the tenant.

```
/foodfast/... → FoodFast  
/quickshop/... → QuickShop
```

This also makes tenant ownership explicit while resolving routes.

4. Actors in the GateKeeper System

GateKeeper has four important types of actors.

```
PLATFORM_ADMIN  
  ↓  
Owns and operates GateKeeper  
  
TENANT_ADMIN  
  ↓  
Manages one organization's Gateway configuration  
  
GATEWAY_SERVICE  
  ↓  
Machine account used by Smart Gateway  
  
TENANT CUSTOMER
```

↓
Uses FoodFast/QuickShop, not GateKeeper Management

Understanding these actors before designing authentication prevents multiple security problems later.

5. Platform Administrator

The `PLATFORM_ADMIN` operates GateKeeper itself.

It can perform operations such as:

- Create organizations
- Create the first administrator for an organization
- Activate organizations
- Suspend organizations
- Soft-delete organizations
- View organizations
- View infrastructure health

For example:

```
Platform Admin
|
+---- Create FoodFast
|
+---- Create QuickShop
```

However, the platform administrator is **not responsible for FoodFast's business operations**.

It should not:

- Place FoodFast orders
- Access customer accounts
- Authenticate FoodFast customers

- Manage FoodFast's restaurant data

GateKeeper manages API infrastructure, not the tenant application's business domain.

6. Tenant Administrator

Each organization has a `TENANT_ADMIN`.

For example:

```
FoodFast
└─ foodfast-admin

QuickShop
└─ quickshop-admin
```

A tenant administrator logs into GateKeeper Management and manages Gateway configuration belonging to **only its own organization**.

It can configure things such as:

- Routes
- Allowed HTTP methods
- Timeouts
- Rate limits
- Idempotency
- Header rules
- Route activation

For example:

```
foodfast-admin
└─ manages FoodFast routes
```

It should never be able to modify:

QuickShop routes

This is the core idea behind **tenant isolation**.

For this project, the model is intentionally simplified to:

1 Organization → 1 Tenant Administrator

Features such as the following are outside the current scope:

- Multiple developers
- Viewer roles
- Invitations
- Billing users
- Subscription plans

7. Gateway Service Account

Smart Gateway also needs to communicate with API Management.

Instead of giving it a human account, we create a machine identity:

GATEWAY_SERVICE

Its job is simple:

```
Smart Gateway
  |
  | asks for active configuration
  ↓
API Management
```

The Gateway Service account can **read configuration**.

It cannot modify it.

For example:

```
Allowed:
GET active routes

Not allowed:
POST route
PUT route
DELETE route
```

This follows the principle of least privilege.

8. Tenant Customers Are Different From Management Users

This is one of the most important security boundaries in the project.

Suppose a FoodFast customer places an order.

That user might have a token issued by FoodFast:

```
FoodFast Customer
  |
  | FoodFast JWT
  ↓
Smart Gateway
  |
  | token passed forward
  ↓
FoodFast Backend
```

GateKeeper does **not** turn that customer into a GateKeeper management user.

The customer does not log into:

```
GateKeeper Management
```

Instead, the customer's credential belongs to FoodFast.

9. Who Validates What?

There are two different types of validation happening.

GateKeeper Validation

GateKeeper is responsible for Gateway policies.

It can ask questions such as:

- Does a route match this request?
- Is the route active?
- Has the rate limit been exceeded?
- Is this HTTP method allowed?
- Does an idempotent POST contain an `Idempotency-Key` ?
- Which headers should be added?
- Which headers should be removed?
- What timeout should be used?

These are **Gateway concerns**.

Tenant Authentication

FoodFast is responsible for questions such as:

- Is this customer logged in?
- Is the JWT valid?
- Can this user access order `123` ?
- Does this customer own the payment method?
- Can this user cancel this order?

These are **application concerns**.

Therefore:

GateKeeper
↓
Validates Gateway policy

FoodFast
↓
Validates FoodFast user identity and authorization

This creates a clean trust boundary.

10. Management Credentials Are Never Customer Credentials

Consider:

```
foodfast-admin
```

This account exists only to configure FoodFast inside GateKeeper.

It is not sent when a customer places an order.

The actual request looks conceptually like:

```
FoodFast Customer  
  |  
  | customer's FoodFast token  
  ↓  
Smart Gateway  
  |  
  ↓  
FoodFast Backend
```

Not:

```
FoodFast Customer  
  |  
  | foodfast-admin credential ❌
```

↓
Smart Gateway

Administrative configuration and customer authentication are completely separate concerns.

11. Designing the Database From First Principles

Instead of starting by creating JPA entities, start with the questions the system needs to answer.

This helps the database structure emerge naturally from the business requirements.

12. Organizations

Question

| Which companies are using GateKeeper?

We need an `organizations` table.

```
organizations
  id
  name
  slug
  status
  created_at
  updated_at
  deleted_at
```

Example:

```
id      = 1
name    = FoodFast
```

```
slug    = foodfast
status  = ACTIVE
```

Why Have Both `name` and `slug` ?

The name is meant for humans.

```
name = FoodFast
```

The slug is a URL-safe identity.

```
slug = foodfast
```

This makes routes like this possible:

```
/foodfast/orders/**
```

13. Organization Status

A simple boolean such as:

```
active = true/false
```

does not describe enough states.

Instead, an organization has a status.

```
PENDING
ACTIVE
SUSPENDED
DELETED
```

PENDING

The organization has been provisioned but is not ready to use the system.

ACTIVE

The organization can use GateKeeper Management and its Gateway configuration.

SUSPENDED

The organization temporarily cannot use the system.

The data still exists and the organization can potentially be reactivated later.

DELETED

The organization has been soft-deleted but its information can still be retained for historical purposes.

This is more expressive than a simple boolean.

14. Management Users and Roles

Question

| Who is allowed to operate the GateKeeper control plane?

We need roles such as:

```
PLATFORM_ADMIN
TENANT_ADMIN
GATEWAY_SERVICE
```

And a table for management identities:

```
management_users
  id
  organization_id
  role_id
  username
  password_hash
  enabled
```

15. Why Is `organization_id` Nullable?

Not every management user belongs to an organization.

For example:

```
platform-admin
  → organization_id = NULL

gateway-service
  → organization_id = NULL

foodfast-admin
  → organization_id = FoodFast

quickshop-admin
  → organization_id = QuickShop
```

Platform-level identities belong to GateKeeper itself.

Tenant administrators belong to a particular organization.

16. One Tenant Administrator Per Organization

For this version of GateKeeper, each organization has only one tenant administrator.

Therefore, a uniqueness constraint can be applied to:

```
management_users.organization_id
```

Conceptually:

```
FoodFast → maximum one tenant admin
QuickShop → maximum one tenant admin
```

Platform-level users still have:

```
organization_id = NULL
```

MySQL allows multiple `NULL` values in this type of unique constraint, so multiple platform-level accounts can coexist while still enforcing one organization-bound administrator.

17. API Routes

Question

| What information does Smart Gateway need to process a request?

That leads to the `api_routes` table.

```
api_routes
  organization_id
  route_id
  name
  path_pattern
  target_base_url
  target_path_prefix
  requests_per_minute
  response_timeout_ms
  idempotency_enabled
  active
  configuration_version
```

A route contains both:

```
Routing information
+
Gateway policy
```

18. Route Identity Is Tenant-Scoped

The important uniqueness rule is:

```
UNIQUE (organization_id, route_id)
```

This means `route_id` does not have to be globally unique.

For example:

```
FoodFast + orders  
QuickShop + orders
```

is perfectly valid.

Both companies can call their route:

```
orders
```

because the organization is part of its identity.

Conceptually:

```
Route Identity  
=  
Organization + Route ID
```

19. Route Allowed Methods

A route may allow multiple HTTP methods.

For example:

```
/orders  
  
GET  
POST  
PUT  
DELETE
```

Trying to store all of them in one route column would make the model harder to manage.

Therefore, allowed methods are represented separately:

```
route_allowed_methods
```

Conceptually:

```
api_route
|
+---- GET
+---- POST
+---- PUT
```

This creates a one-to-many relationship.

20. Route Header Rules

A route can also have multiple header rules.

For example:

```
Add X-Tenant-ID
Remove X-Internal-Header
Add X-Gateway-Version
```

Therefore, these rules are stored separately:

```
route_header_rules
```

Conceptually:

```
api_route
|
+---- header rule
```

```
+---- header rule
+---- header rule
```

Both allowed methods and header rules are children of the live route.

If the live route is deleted, these live child records can also be deleted.

21. Route Configuration Versioning

Now consider another problem.

Suppose a route changes multiple times.

```
Version 1
  ↓
Version 2
  ↓
Version 3
```

If we only update `api_routes`, the previous configuration disappears.

But configuration history can be extremely useful.

So we create:

```
route_configuration_versions
```

Containing information such as:

```
organization_id
route_id
version_number
change_type
configuration_snapshot
created_at
```

22. Every Route Mutation Creates a Version

A possible history may look like:

```
CREATE
  → Version 1
  → CREATED

PUT
  → Version 2
  → UPDATED

Deactivate
  → Version 3
  → DEACTIVATED

Activate
  → Version 4
  → ACTIVATED

DELETE
  → Version 5
  → DELETED
```

This gives us a historical record of how Gateway configuration evolved.

23. Why Configuration History Should Survive Route Deletion

A configuration version should not depend on the live route continuing to exist.

Suppose:

```
/orders
```

is deleted.

We may still need to know:

```
What configuration existed before deletion?
```

Therefore, `route_configuration_versions` deliberately does not depend on the live route through a foreign key that would cause its history to disappear when the route is removed.

The live state and historical state have different lifecycles.

```
Live Route
  → may be deleted

Route History
  → should remain
```

24. Control Plane Auditing

Versioning answers one important question:

| What did the route configuration look like?

But it does not completely answer:

| Who performed the operation?

For that, GateKeeper needs auditing.

We create:

```
control_plane_audit_records
```

with information such as:

```
organization_id
actor
action
resource_type
resource_id
correlation_id
duration_ms
result
```

```
details
created_at
```

An audit record can tell us things such as:

```
Actor      : foodfast-admin
Action     : UPDATE_ROUTE
Resource   : orders
Result     : SUCCESS
Duration   : 31 ms
```

25. Versioning vs Auditing

These two concepts are related but solve different problems.

Versioning

Answers:

```
What did the configuration look like?
```

For example:

```
Version 3 of the FoodFast orders route
```

Auditing

Answers:

```
Who attempted which operation, and what happened?
```

For example:

```
foodfast-admin attempted UPDATE_ROUTE
```

Result: SUCCESS

A useful mental model is:

```
Version History
  ↓
State history

Audit History
  ↓
Operator/action history
```

Both are valuable, but they should not be confused.

26. Final Database Relationship Model

The simplified relationship model becomes:

```
organizations
|
+---- 0..1 management_users
|
+---- many api_routes
|
+---- many route_allowed_methods
|
+---- many route_header_rules

organizations
|
+---- many route_configuration_versions
|
+---- many control_plane_audit_records
```

At this point, the database structure reflects the major business concepts instead of being a collection of arbitrary tables.

27. Design APIs Before Writing Controllers

Once the actors, ownership model, and database relationships are clear, we can design the API contract.

A good API should make two things obvious:

```
Who is making the request?  
+  
What operation are they allowed to perform?
```

Controllers should come **after** these boundaries are understood.

28. Public APIs

The publicly accessible management APIs begin with:

```
GET /health  
  
POST /api/auth/login
```

Health Endpoint

```
GET /health
```

Allows us to check whether API Management is running.

Login Endpoint

```
POST /api/auth/login
```

Allows GateKeeper management users to authenticate.

These users include identities such as:

```
PLATFORM_ADMIN  
TENANT_ADMIN
```

GATEWAY_SERVICE

depending on how authentication is configured.

29. Why There Is No Public Registration API

GateKeeper does not provide an endpoint such as:

```
POST /api/auth/register
```

The reason is architectural.

GateKeeper manages **management accounts**, not customer accounts.

Organizations and their administrators are provisioned through GateKeeper's controlled management flow.

A random user should not be able to register themselves as:

```
TENANT_ADMIN
```

or:

```
PLATFORM_ADMIN
```

Similarly, FoodFast customers do not register with GateKeeper.

Their accounts belong to FoodFast.

```
FoodFast Customer Registration
  ↓
FoodFast

GateKeeper Management Account Creation
  ↓
GateKeeper
```

These are two completely different identity systems.

30. Overall Request and Configuration Flow

The control-plane flow currently looks like:

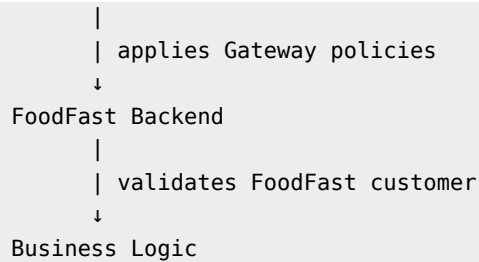
```
Platform Administrator
  |
  | provisions
  ↓
FoodFast
  |
  | gets
  ↓
FoodFast Tenant Admin
  |
  | configures routes
  ↓
API Management Service
  |
  | stores configuration
  ↓
MySQL
```

Later, Smart Gateway can consume that configuration:

```
Smart Gateway
  |
  | GATEWAY_SERVICE identity
  ↓
API Management
  |
  | active route configuration
  ↓
Smart Gateway
```

Actual customer traffic follows a different path:

```
FoodFast Customer
  |
  | FoodFast credential
  ↓
Smart Gateway
```

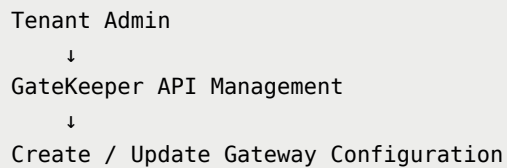


This separation is the foundation of the GateKeeper architecture.

31. Control Plane vs Customer Traffic

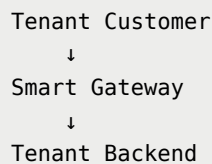
It is important not to mix these two flows.

Management Flow



Authentication here belongs to **GateKeeper**.

Customer Flow



Authentication of the end user ultimately belongs to the **tenant application**.

This separation allows GateKeeper to manage infrastructure policies without taking ownership of every tenant's customer identity system.

32. Key Design Principles

1. A Microservice Must Own a Clear Problem

Creating separate Spring Boot applications does not automatically create a good architecture.

Each service needs a clear responsibility.

```
API Management → configuration
Smart Gateway  → request processing
Analytics       → observations
```

2. Separate Control Plane From Data Plane

Configuration management and live request processing solve different problems.

Keeping them separate makes the architecture easier to evolve.

3. Tenant Identity Must Be Explicit

Routes belong to organizations.

```
organization_id + route_id
```

Together identify a route.

This prevents one tenant's configuration from accidentally becoming another tenant's configuration.

4. Management Users Are Not Application Customers

```
foodfast-admin
```

and:

FoodFast customer

are completely different identities.

Mixing these concepts would create both architectural and security problems.

5. The Gateway Enforces Gateway Policy

GateKeeper deals with things such as:

- Routing
- Rate limiting
- Timeouts
- Headers
- Idempotency
- Route activation

The tenant backend continues to own application-level authorization.

6. Design the Database From Business Questions

Instead of starting with entities, ask:

- Which companies use the system?
- Who can operate it?
- What configuration does the Gateway need?
- What values can repeat?
- How do we retain configuration history?
- Who changed the configuration?

The tables then emerge naturally from the answers.

7. Live State and Historical State Have Different Lifecycles

Deleting a route should not automatically destroy its configuration history.

Historical data exists for a different purpose than the current live configuration.

8. Versioning and Auditing Are Different

Versioning

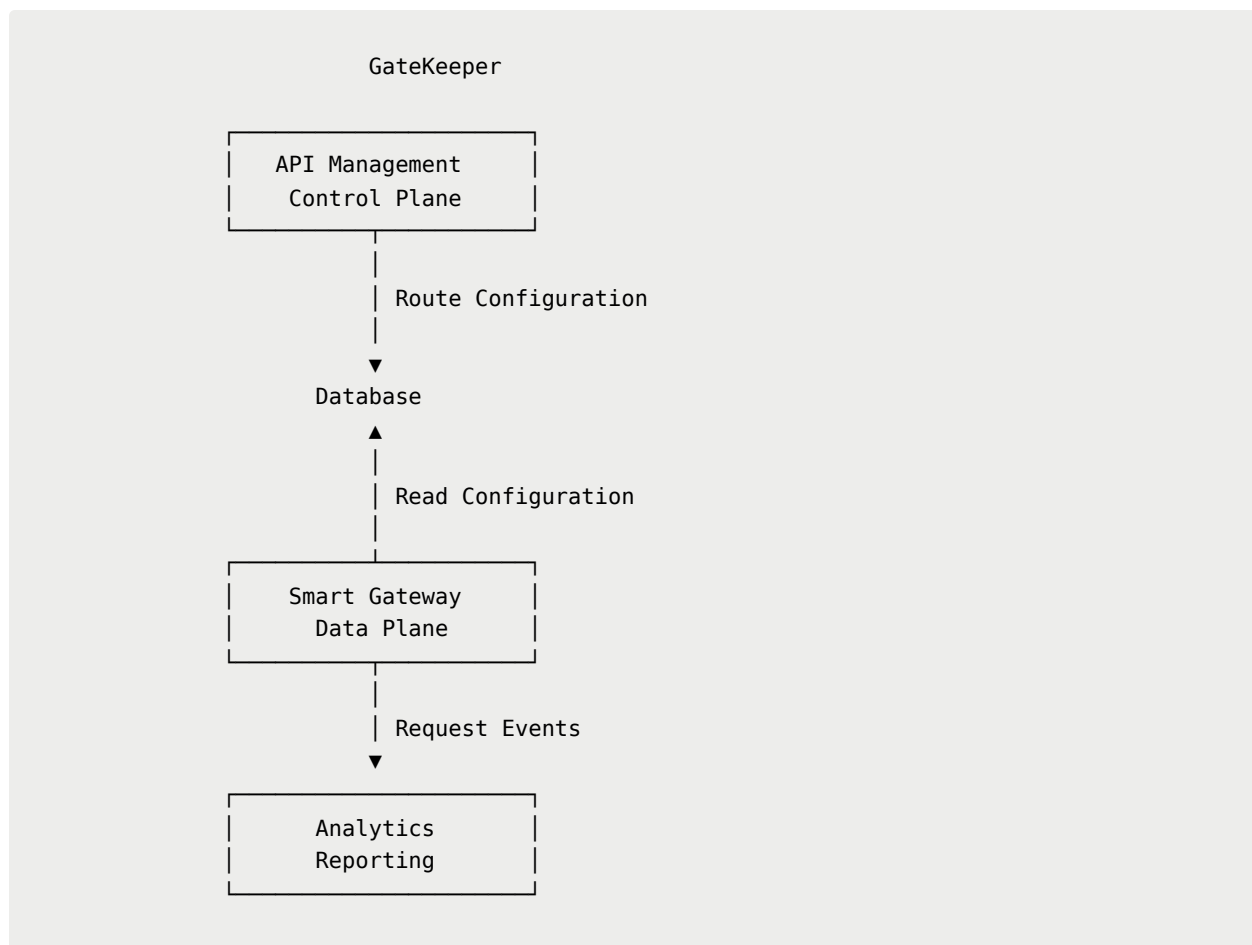
→ What did the configuration look like?

Auditing

→ Who performed the operation and what happened?

A mature control plane usually needs both.

Final Architecture So Far



The important idea is that **API Management defines the behaviour of the Gateway, while Smart Gateway applies that behaviour to real traffic.**

With the ownership model, tenant boundaries, management identities, route model, version history, auditing, and API contract defined, the control plane now has a clear architectural foundation.

Coder Army